Fundamentals of Artificial Intelligence, Machine and Deep Learning Study Sheet

Amina Hanid - Sarra Lajnef
Based on: Course FAI-polito

April 21, 2026

Contents

1 Lecture 1: Introduction to AI, ML and DL **3**

2 Lecture 2: Probability and Statistics Bootcamp **5**
2.1 The Product Chain Rule . . . 5
2.2 Independence . . . 5
2.3 Bayes' Rule . . . 5
2.4 The Naïve Bayes Classifier: From Decision to Probability . . . 5
2.5 Parameter Estimation . . . 7
2.6 Estimator Properties . . . 7
2.7 Bias-Variance Decomposition: . . . 8
2.8 Fundamental Laws . . . 9
2.8.1 Law of Large Numbers (LLN) . . . 9
2.8.2 Central Limit Theorem (CLT) . . . 9
2.8.3 Applications in Machine Learning . . . 10

3 Lecture 3: How to Build a Deep Learning Pipeline **11**
3.1 Defining Objectives & Computer Vision Tasks . . . 11
3.2 Data Collection and Preprocessing . . . 11
3.3 Model Design . . . 11
3.4 Training and Evaluation . . . 11
3.5 Improving through Experimentation . . . 12
3.6 Final Step: Save Your Model . . . 12

4 Lecture 4 : Unsupervised Learning **21**
4.1 Clustering . . . 21
4.2 The K-means Clustering Problem . . . 21
4.2.1 Problem Definition . . . 21
4.2.2 Computational Complexity . . . 21
4.2.3 Voronoi Diagrams in K-means . . . 21
4.2.4 Challenges: Initialization and Local Minima . . . 22
4.2.5 What is K-means optimizing? . . . 23
4.2.6 K-means as Coordinate Descent . . . 23
4.3 Gaussian Mixture Mode . . . 24
4.3.1 Density Estimation: Generative Approach . . . 24
4.3.2 Partitioning Algorithms . . . 24
4.4 Gaussian Mixture Models (GMM) . . . 24
4.4.1 The Generative Process . . . 25
4.4.2 Probabilistic Clustering: Mixture of Gaussians . . . 25
4.4.3 Maximum Likelihood Estimation (MLE) . . . 26
4.5 From K-means to General GMM . . . 26
4.5.1 K-means as a Special Case of GMM . . . 26
4.5.2 General GMM (Anisotropic Gaussians) . . . 26
4.5.3 Decision Boundaries . . . 26

4.5.4 Estimation: Why we use EM . . . 27
4.5.5 General GMM: Maximum Likelihood Estimation (MLE) . . . 27
4.5.6 Comparison of Models . . . 27
4.6 Introduction to EM . . . 28
4.6.1 Simple vs. General GMMs . . . 28
4.7 K-Nearest Neighbors (KNN) in Supervised Learning . . . 29
4.7.1 Distance Metrics . . . 29
4.7.2 kNN Pros and Cons . . . 29
4.7.3 kNN Tricks (Reduce Complexity) . . . 30
4.8 Non-linearity and Feature Maps . . . 31
4.8.1 Example: Circular Separation . . . 31
4.9 Computational Efficiency Problem . . . 31
4.10 The Solution: Implicit Computation . . . 31
4.11 Feature Engineering and Nonlinear Mappings . . . 31
4.11.1 The Core Concept . . . 31
4.11.2 Practical Applications . . . 31

5 Lecture 5: Multi-Layer Perceptrons and Optimization 33
5.1 Artificial Neurons and the Perceptron . . . 33
5.1.1 Mathematical Model . . . 34
5.2 Error Function: Mean Squared Error (MSE) . . . 34
5.3 Gradient Descent Intuition . . . 34
5.4 Incremental (Stochastic) Gradient Descent . . . 34
5.5 Limitations: The XOR Problem . . . 35
5.6 Multi-Layer Perceptron (MLP) . . . 35
5.7 Training and Backpropagation . . . 35
5.7.1 Backpropagation Algorithm . . . 35
5.8 Hierarchical Representations in Deep Learning . . . 35
5.9 Optimization Methods . . . 36
5.9.1 Stochastic Gradient Descent (SGD) . . . 36
5.9.2 Advanced Optimizers . . . 36
5.10 Optimization Summary . . . 37
5.11 The Core Concept: Large Margin Separation . . . 39
5.11.1 Mathematical Formulation . . . 39
5.12 The Dual Problem and Support Vectors . . . 39
5.12.1 Key Properties . . . 39
5.13 Soft Margin and Slack Variables . . . 40
5.14 Nonlinear Separation: The Kernel Trick . . . 40
5.14.1 The Kernel Perceptron . . . 41
5.14.2 Common Kernel Functions . . . 41
5.14.3 Kernel Conditions and Mercer's Theorem . . . 41
5.14.4 Mathematical Properties . . . 42

6 Lecture 6: 43
6.1 Logistic Regression for Classification . . . 43
6.1.1 Maximum Likelihood Estimation (MLE) . . . 43
6.1.2 Gradient Ascent for Logistic Regression . . . 44
6.2 Regularization in Logistic Regression . . . 45
6.3 Classifier Calibration . . . 46
6.3.1 Calibration Metrics and Visualization . . . 46
6.3.2 Calibration Methods (Post-processing) . . . 47

1 Lecture 1: Introduction to AI, ML and DL

1. What is AI? Approaches to Artificial Intelligence

AI is broadly defined as making computers think like a human. The field looks at this through different lenses:

Thinking Humanly:

Acting Humanly (The Turing Test Approach):

- **Natural language processing:** to communicate and generate comprehensible sentences.
- **Knowledge representation:** to store knowledge.
- **Automated reasoning:** to answer questions and draw conclusions.
- **Machine learning:** to detect/extract patterns, adapt to new circumstances, and improve effective behavior.
- **Computer vision:** to perceive objects.
- **Robotics:** to manipulate objects and interact beyond language.

Issues with the Turing Test:

- Success depends on deception.
- Protocols and judges vary.
- Chatbots can perform well using "cheap tricks".

The Winograd Schema: Designed to overcome Turing Test weaknesses. These are multiple-choice questions that people can answer easily, but computers cannot answer using simple "cheap tricks".

Thinking Rationally (The Logicist Approach):

Acting Rationally (AI = Computational Rationality):

2. Machine Learning Foundations

Definition of Learning: An agent learns when its performance (P) at a specific task (T) improves with experience/data (E). We want the computer to do well without manually building its competence.

Classical Programming vs. Machine Learning: **Classical Programming:** You input Data + Program (Rules) into the computer, and it produces the Output.

Machine Learning (Programming with Data): Rule-based implementation is brittle, difficult to maintain, and misses edge cases. Instead, we input Data + Output (Answers), and the learning algorithm produces the Program (Rules/Model).

ML Pseudo-Algorithm:

1. Understand domain, prior knowledge, and goals.
2. Data integration, selection, cleaning, and pre-processing.
3. Learn models.
4. Interpret results.
5. Consolidate and deploy discovered knowledge.

3. The AI / ML / DL Hierarchy

AI vs. ML vs. DL:

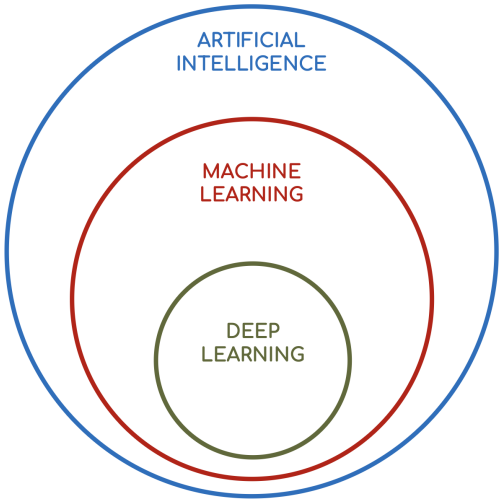


Figure 1: AI vs. ML vs. DL

Main ML Paradigms:

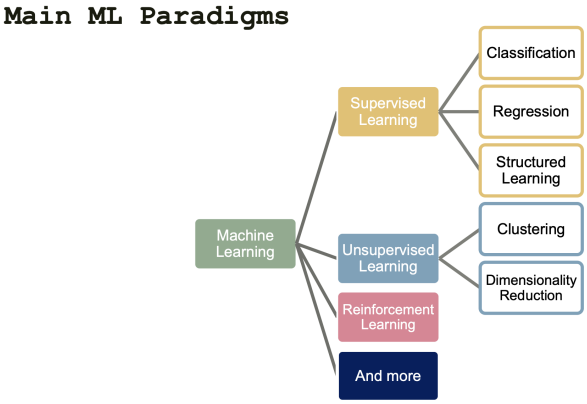


Figure 2: Main ML Paradigms

2 Lecture 2: Probability and Statistics Bootcamp

2.1 The Product Chain Rule

Product Rule:

$$P(y)P(x|y) = P(x, y)$$

Chain Rule (General Form):

$$P(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} P(x_i|x_1, \ldots, x_{i-1})$$

Chain Rule (Three Variables):

$$P(x_1, x_2, x_3) = P(x_1)P(x_2|x_1)P(x_3|x_1, x_2)$$

2.2 Independence

Independent Random Variables (Joint):

$$P(X, Y) = P(X)P(Y)$$

Independent Random Variables (Conditional):

$$P(X|Y) = P(X)$$

2.3 Bayes' Rule

Bayes' Rule:

$$P(x|y) = \frac{P(y|x)}{P(y)}P(x)$$

Bayesian Proportionality (MAP:Maximum a Posteriori): P(y) = constant (because given) tikz amsmath

likelihood

$$P(x|y) \propto \frac{P(y|x)}{P(y)}\cancel{P(x)}$$

Ignored in f_{NB}

posterior **prior**

2.4 The Naïve Bayes Classifier: From Decision to Probability

The Naïve Bayes classifier is a generative learning model based on Bayes' Theorem, utilizing a strong simplifying assumption of independence.

1. The "Naïve" Assumption

It is assumed that each attribute X_i is conditionally independent of all other attributes given the class label Y. This simplifies the joint likelihood into a product of individual likelihoods:

$$P(X_1, \ldots, X_d|Y) = \prod_{i=1}^{d} P(X_i|Y)$$

2. The Decision Rule: $f_{NB}(x)$

The primary goal of the classifier is to predict the class label. To achieve this, we seek the class y that maximizes the product of the prior and the likelihood.

The decision function $f_{NB}(x)$ is defined as:

$$f_{NB}(x) = \arg\max_y \left[P(y) \prod_{i=1}^{d} P(x_i|y) \right]$$

Note: The denominator $P(x)$ is ignored here because it is identical for all classes and does not change the choice of the "winner".

3. Calculating the Posterior Probability: $P(y|x)$

If we wish to know the level of certainty (the actual probability between 0 and 1), we must normalize the scores. The complete formula is:

$$P(Y = y_k|X) = \frac{P(y_k) \prod_{i=1}^{d} P(x_i|y_k)}{P(x)}$$

4. The Normalization Factor α

In practice, $P(x)$ is often difficult to calculate directly. We use a normalization factor α to ensure the sum of probabilities across all classes equals 1:

1. **Calculate the unnormalized score** for each class k:

$$\tilde{P}_k = P(y_k) \prod_{i=1}^{d} P(x_i|y_k)$$

2. **Calculate** α (the inverse of the sum of all scores):

$$\alpha = \frac{1}{\sum_k \tilde{P}_k}$$

3. **Final Probability**:

$$P(y_k|x) = \alpha \tilde{P}_k$$

Summary of Steps

- **Step 2 (Decision)**: We compare the scores $\tilde{P}_k$. The largest determines $f_{NB}(x)$. This is the **action**.
- **Step 4 (Normalization)**: We use α to transform these scores into probabilities $P(y|x)$. This is the **measure of confidence**.

5. Naïve Bayes Classifier Summary:

Naïve Bayes Decision Rule:

Given:

- Class prior P(y)
- d conditionaly independant features $X_1, ..., X_d$ given the class label Y
- For each X_i feature, we have the conditional likelihood $P(X_i|Y)$

$$f_{NB}(x) = \arg\max_y P(y) \prod_{i=1}^{d} P(x_i|y)$$

- **Computationally very fast**
  - Training: only one pass over the training set
  - Classification: linear in the number of attributes
- Despite its conditional independence assumption, Naïve Bayes classifier shows a good performance in several application domains.

When to use?

- A moderate or large training set available with instances represented by a large number of attributes.

Issue: It might happen that an attribute never appears for a specific class, e.g., `Temperature=cold` never appears together with `EnjoySpt=yes` $\longrightarrow P(\text{cold}|\text{yes}) = 0 \longrightarrow P(\text{yes}|\text{cold}) = 0$.

Laplacian Correction: A trick used if a specific attribute never appears with a specific class in the training data (which would yield a 0 probability). It adds "virtual" samples to prevent the entire probability product from becoming zero. Add one or more "virtual" samples with (`cold`, `yes`) and one with (`warm`, `yes`).

$$\hat{P}(X_j = a_{jk} \mid C = c_i) = \frac{n_c + mp}{n + m}$$

n_c : number of training examples for which $X_j = a_{jk}$ and $C = c_i$

n : number of training examples for which $C = c_i$

p : prior estimate (usually, $p = 1/t$ for t possible values of X_j)

m : weight to prior (number of "virtual" examples, $m \geq 1$)

Figure 3: laplacien

2.5 Parameter Estimation

Maximum Likelihood Estimation (MLE):(page 36 lecture 2 Intro proba)
"Given a model, which parameter best explains the data I saw?" + No prior knowledge is considered

$$\hat{\theta}_{MLE} = \arg\max_{\theta} P(D|\theta)$$

Maximum a Posteriori (MAP) Estimation:
It also considers prior beliefs about the parameter

$$\hat{\theta}_{MAP} = \arg\max_{\theta} P(D|\theta)P(\theta)$$

MLE for Gaussian Mean:

$$\hat{\mu}_{MLE} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

MLE for Gaussian Variance:

$$\hat{\sigma}^2_{MLE} = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{\mu})^2$$

2.6 Estimator Properties

Bias: how close is the estimate to the true value ?
Variance: how much does the estimate change for different runs (eg. different datasets)?
Bias of an Estimator:

$$bias(\hat{\theta}_m) = E[\hat{\theta}_m] - \theta$$

The estimator is **unbiased** if $bias(\hat{\theta}_m) = 0$
Estimator of Gaussian Variance: (page 47 lecture 2 intro proba)

$$\hat{\sigma}^2_m = \frac{1}{m-1}\sum_{i=1}^{m}(x^{(i)} - \hat{\mu}_m)^2$$

The Variance of an Estimator

1. Definition and Intuition

The variance indicates how much we expect the estimator to vary as a function of different data samples from the same distribution.

2. Efficiency and Comparison

If two estimators of a parameter are both unbiased, the better one is the one with the least amount of variability.

- An estimator $\hat{\theta}_1$ is said to be **more efficient** than $\hat{\theta}_2$ if $Var(\hat{\theta}_1) < Var(\hat{\theta}_2)$.

3. Standard Error (SE)

The square root of the variance is called the **standard error**, denoted as $SE(\hat{\theta})$.

- It measures how we expect the estimate to vary when obtaining different samples from the same distribution.
- **Standard Error of the Mean:** For a dataset of size m, it is given by:

$$SE(\hat{\mu}_m) = \sqrt{Var\left[\frac{1}{m}\sum_{i=1}^{m} x^{(i)}\right]} = \frac{\sigma}{\sqrt{m}}$$

- *Note:* As the number of samples m increases, the accuracy of the prediction improves as the SE decreases.

4. Confidence Intervals and Decision Making

Since the mean is normally distributed (due to the Central Limit Theorem), we can compute the probability that the true expectation falls in a specific interval.

- **95% Confidence Interval:** Centered on the mean $\hat{\mu}_m$ is given by:

$$(\hat{\mu}_m - 1.96SE(\hat{\mu}_m), \hat{\mu}_m + 1.96SE(\hat{\mu}_m))$$

- **Comparison Rule:** Algorithm A is better than B if the error upper bound of A is less than the lower bound of B.

Mean Squared Error (MSE):

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2]$$

2.7 Bias-Variance Decomposition:

Let $\hat{\theta}$ be an estimator for an unknown parameter θ. The **Mean Squared Error (MSE)** of this estimator can be decomposed into the square of the bias and the variance:

Formula:

$$MSE(\hat{\theta}) = b^2(\hat{\theta}) + Var(\hat{\theta})$$

Proof:

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2]$$

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}] + \mathbb{E}[\hat{\theta}] - \theta)^2]$$

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2 + (\mathbb{E}[\hat{\theta}] - \theta)^2 + 2(\hat{\theta} - \mathbb{E}[\hat{\theta}])(\mathbb{E}[\hat{\theta}] - \theta)]$$

$$MSE(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2] + (\mathbb{E}[\hat{\theta}] - \theta)^2 + 2\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])](\mathbb{E}[\hat{\theta}] - \theta)$$

- The first term is the definition of Variance: $\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2] = Var(\hat{\theta})$.
- The second term is the square of the Bias: $(\mathbb{E}[\hat{\theta}] - \theta)^2 = b^2(\hat{\theta})$.
- The third term vanishes because $\mathbb{E}[\hat{\theta} - \mathbb{E}[\hat{\theta}]] = \mathbb{E}[\hat{\theta}] - \mathbb{E}[\hat{\theta}] = 0$.

Result:

$$MSE(\hat{\theta}) = Var(\hat{\theta}) + b^2(\hat{\theta})$$

Comparison: Generative vs. Discriminative Models

Feature	**Generative Models (MAP)**	**Discriminative Models (MLE)**
Goal	Estimate the joint probability $P(X, Y)$.	Estimate the conditional probability $P(Y\|X)$ directly.
Approach	Model how the data was generated: $P(X\|Y)P(Y)$.	Model the decision boundary between classes.
Bayes' Rule	Essential to infer $P(Y\|X)$ from the learned joint distribution.	Not required, as $P(Y\|X)$ is learned directly.
Prior Knowledge	Easy to incorporate via prior distributions $P(Y)$ or $P(\theta)$.	Harder to incorporate prior information about the data distribution.
Data Complexity	Struggles with very complex distributions (e.g., high-res images).	Superior for complex data like text, images, or movies.
Small Samples	Generally more robust (Bayesian/MAP approach is "better when samples are small").	May perform poorly or overfit with very few samples.
Bias / Variance	**High Bias** (due to strong assumptions like Naïve Bayes) / **Low Variance**.	**Low Bias** / **Higher Variance** (requires more data to stabilize the boundary).
Example	Naïve Bayes.	Logistic Regression, SVM, Neural Networks.

2.8 Fundamental Laws

2.8.1 Law of Large Numbers (LLN)

The Law of Large Numbers states that as the number of identically distributed, independent variables (i.i.d.) increases, their sample mean converges to the true theoretical mean of the population.

Definition

Let $X_1, X_2, \ldots, X_n$ be a sequence of i.i.d. random variables with a finite expected value $E[X_i] = \mu$. The sample mean is defined as:

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^{n} X_i$$

As $n \to \infty$, the sample mean $\bar{X}_n$ converges to the population mean μ:

$$\bar{X}_n \xrightarrow{p} \mu$$

2.8.2 Central Limit Theorem (CLT)

The Central Limit Theorem describes the *distribution* of the sample mean. It states that, regardless of the original distribution of the variables (as long as it has finite variance), the distribution of the sample mean will approach a Normal (Gaussian) distribution as the sample size becomes large.

Theorem Details

Given a set of n i.i.d. random variables with mean μ and variance σ^2, the distribution of their sum (or mean) tends toward:

$$Z_n = \frac{1}{\sqrt{n}} \sum_{i=1}^{n} \frac{X_i - \mu}{\sigma} \approx \mathcal{N}(0, 1)$$

Standard Error

The standard deviation of this sample mean distribution is known as the **Standard Error (SE)**:

$$SE(\bar{X}_n) = \frac{\sigma}{\sqrt{n}}$$

This formula shows that the estimate becomes more precise (less variable) as the dataset size n increases.

2.8.3 Applications in Machine Learning

- **Error Bounds:** Since the mean is normally distributed per the CLT, we can calculate the probability that a true expectation falls within a specific chosen interval.
- **Model Comparison:** An algorithm A is considered better than B if the error upper bound of A is strictly less than the lower bound of B.

3 Lecture 3: How to Build a Deep Learning Pipeline

Introduction A deep learning pipeline is a structured workflow designed to ensure meaningful and reproducible results. It follows a logical progression:

Define Task → Data Prep → Model Selection → Training & Evaluation → Experimentation → Deployment

3.1 Defining Objectives & Computer Vision Tasks

Deep learning models process data through **numerical encoding** to learn representations, patterns, and features.

Common Computer Vision Tasks

- **Image Classification**: Assigning a single label to an entire image (e.g., "Cat").
- **Object Detection**: Identifying objects and determining their location via bounding box coordinates.
- **Semantic Segmentation**: Identifying pixels belonging to each class (pixel-wise mask).
- **Instance Segmentation**: Distinguishing between individual objects of the same class (unique masks for different instances).

3.2 Data Collection and Preprocessing

Understanding Data Nature

- **Supervision Type**: Supervised (labeled), Unsupervised (unlabeled), or Semi-supervised (mixed).
- **Data Imbalance**: Overrepresented categories cause bias. *Solution*: Resampling or class balancing.
- **Dataset Size**: Small datasets lead to **overfitting** (memorization). *Solution*: Data Augmentation (flipping, rotating) or synthetic data.

Preprocessing & Normalization

Normalization ensures data is on a standard scale for stable learning. Images must also be processed to have fixed dimensions through resizing and cropping.

The Two Key Data Splits

The ultimate goal is **generalization** to unseen data.

- **Training Set**: Used for model learning; should be large and diverse.
- **Test Set**: Used only for final evaluation; must never be seen during training.

3.3 Model Design

Selection depends on:

- **Task Type**: (e.g., Classification vs. Segmentation).
- **Data Size**: Pretrained models for small data; training from scratch for large data.
- **Resources**: Memory and processing power (GPU/TPU).
- **Inference Speed**: Lightweight models are required for real-time robotics.

3.4 Training and Evaluation

- **Training**: Iterative update of parameters using a **loss function** until the error is minimized.
- **Evaluation**: Testing generalization to ensure the model hasn't just memorized the training data.

Common Metrics

- **Accuracy**: Percentage of correct predictions.
- **Precision & Recall**: Handling false positives vs. false negatives.
- **IoU (Intersection over Union)**: Measures overlap between predicted and ground truth masks/boxes.

3.5 Improving through Experimentation

Hyperparameters

Settings chosen before training begins:

- **Learning Rate**: Size of the parameter updates at each step.
- **Epochs**: Number of complete passes through the dataset.
- **Batch Size**: Samples processed before each parameter update.

Validation Strategies

- **Validation Set**: A separate set used to tune hyperparameters to avoid overfitting the test set.
- **K-Fold Cross-Validation**:
  1. Split data into K equal folds.
  2. Train K times, using a different fold as validation each time.
  3. Average the performance for a reliable estimate.

3.6 Final Step: Save Your Model

Ensure the trained parameters and architecture are stored for deployment.

Lecture 3bis: Intro to Learning Theory

Supervised Learning Foundations

- **Input/Output Spaces**: $\mathcal{X}$ (features) and $\mathcal{Y}$ (labels).
- **Joint Distribution D**: $P(X, Y)$ represents the unknown true process.
- **Hypothesis**: $h \in \mathcal{H}$ which is the classifier, where $h : \mathcal{X} \to \mathcal{Y}$.
- We define **the loss of the classifier** that mesures the error between the assigned prediction and the prediction of the correct lableling function $f : \mathcal{X} \to \mathcal{Y}$ where $f(\mathcal{X}) = \mathcal{Y}$
  L(h(X), f(X)) = L(h(X), Y) = $\begin{cases} 1, & \text{if } h(X) \neq Y \\ 0, & \text{if } h(X) = Y \end{cases}$ "0-1 Loss"

Risk Definitions

Expected Risk (True Risk): The True Risk of h is the probability that it does not predict the correct label on a random data point drawn from $\mathcal{D}$.

$\mathrm{R}_{\mathcal{D}}(h) := \mathbb{P}[h(X) \neq f(X)] = \mathbb{P}[h(X) \neq Y]$
$R_{\mathcal{D}}(h) = \mathbb{E}_{(x,y)\sim P}[\ell(h(x), y)]$

Empirical Risk (Training Risk):

$$\hat{L}_n(h) = \frac{1}{n} \sum_{i=1}^{n} \ell(h(x_i), y_i)$$

Empirical Risk Minimization (ERM): The learner chooses $\hat{h} = \arg\min_{h \in \mathcal{H}} \hat{L}_n(h)$.

Generalization and Complexity

VC Dimension (d_{VC}): Measures the capacity of $\mathcal{H}$. For large n, with probability $1-\delta$:

$$L(h) \leq \hat{L}_n(h) + \mathcal{O}\left(\sqrt{\frac{d_{VC}\log(n) + \log(1/\delta)}{n}}\right)$$

Bias-Variance Decomposition

For Mean Squared Error, the error is decomposed as:

$$MSE(\hat{\theta}) = \underbrace{(\mathbb{E}[\hat{\theta}] - \theta)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]}_{\text{Variance}}$$

Structural Risk Minimization (SRM)

To prevent overfitting, we add a regularization term $\Omega(h)$ to the empirical risk:

$$h^* = \arg\min_{h \in \mathcal{H}} \hat{L}_n(h) + \lambda\Omega(h)$$

Bayes Classifier

- Assume $\mathcal{X}$ is discrete for now
- Suppose $(x, y) \sim \mathcal{D}$
- Want to find $h : \mathcal{X} \to \mathcal{Y}$ that minimizes

$$\mathbb{P}[h(x) \neq y] = \sum_{x \in X} \mathbb{P}[h(X) \neq y | X = x] \cdot \mathbb{P}[X = x]$$

- For every $x \in \mathcal{X}$, set $h(x)$ to minimize $\mathbb{P}[h(X) \neq Y | X = x]$
- $\mathbb{P}[h(X) \neq Y | X = x] = 1 - \mathbb{P}[h(X) = Y | X = x]$
- So, to minimize $\mathbb{P}[h(X) \neq Y | X = x]$, maximize $\mathbb{P}[h(X) = Y | X = x]$
- i.e. choose

$$h(x) = \arg\max_{y \in Y} \mathbb{P}[Y = y | X = x]$$

- **Bayes (Optimal) Classifier** (this is NOT Naïve Bayes!)

Method	Category	Optimization Goal	Key Distinction
MLE	Parameter Estimation	Maximize Likelihood: $P(D\|\Theta)$	Assumes no prior knowledge; purely data-driven.
MAP	Parameter Estimation	Maximize Posterior: $P(\Theta\|D) \propto P(D\|\Theta)P(\Theta)$	Incorporates a prior as a form of regularization to prevent overfitting.
Bayes Optimal	Decision Rule	Minimize True Risk: $R_{\mathcal{D}}(h) = \mathbb{E}[L(h(X), Y)]$	Theoretical "gold standard"; uses full joint distribution without independence assumptions.
Naïve Bayes	Practical Classifier	Maximize: $P(y)\prod_{i=1}^{d} P(x_i\|y)$	Simplifies the problem by assuming conditional independence between features.

Table 1: Comparison of MLE, MAP, Bayes Optimal, and Naïve Bayes.

- **MLE vs. MAP**: While MLE only looks at the training data S , MAP introduces a regularization term (the prior). This is crucial when the number of samples is small relative to the number of parameters.
- **Parameter Estimation vs. Classification**: MLE and MAP are used to find "good" model parameters Θ. In contrast, a Bayes Classifier is the strategy used to make a prediction y for a new sample x once those parameters are known.
- **Bayes Optimal vs. Naïve Bayes**: The Bayes Optimal classifier is the theoretical limit of performance (Bayes Risk). However, calculating the full joint probability is often impossible. Naïve Bayes makes the "naïve" assumption that features are independent to make the computation feasible.

Bayes Risk

- Denote by h^* the Bayes classifier
- For the Bayes Risk $R_{\mathcal{D}}(h^*)$ it holds $R_{\mathcal{D}}(h^*) \leq R_{\mathcal{D}}(h)$ for any H and $h \in \mathcal{H}$

Batch Learning

- In practical cases we do not know f and we need to estimate h from the data
- **Learner's input:** Training data, $S = \{(x_1, y_1) \dots (x_m, y_m)\} \in (\mathcal{X} \times \mathcal{Y})^m$
- **Learner's output:** A prediction rule, $h : \mathcal{X} \to \mathcal{Y}$
- **Goal:** h should be correct on future examples

IID condition

- **Key assumption:** Data $(x_1, y_1), \dots, (x_n, y_n)$ are identically and independently distributed (i.i.d.) random labeled examples with distribution $\mathcal{D}$
- This assumption is the connection between what we've seen in the past to what we expect to see in the future

Empirical Risk:

Calculate empirical risk by benchmarking prediction against ground truth

$$R_S(h) = \frac{1}{m} \sum_{i=1}^{m} \mathbb{I}[h(x_i) \neq y_i]$$

Can any algorithm always provide a non-trivial guarantee on the error of the predictor it returns? Can we prove, for a fact, that this AI will actually work on data it hasn't seen before? ı**No:**some assumptions are required

Probably Approximately Correct (PAC) Learning

- Learner doesn't know D and the Bayes predictor.
- Learner receives accuracy ϵ and confidence δ
- Learner can ask for training data S containing m(ϵ, δ) examples
- Learner should output h such that:

$$\mathbb{P}[R_{\mathcal{D}}(h) \leq \epsilon] \geq 1 - \delta$$

- PAC = **Probably** (prob $\geq 1 - \delta$) **Approximately** (accuracy ϵ) **Correct**

Definition (PAC learnability)

A hypothesis class $\mathcal{H}$ is PAC learnable if there exists a function $m_{\mathcal{H}} : (0,1)^2 \to \mathbb{N}$ and a learning algorithm with the following property:

- for every $\epsilon, \delta \in (0,1)$
- for every distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$ such that the realizability assumption holds in $\mathcal{H}$

when running the learning algorithm on $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ i.i.d. examples generated by $\mathcal{D}$, the algorithm returns a hypothesis h such that, with probability of at least $1 - \delta$ (over the choice of the examples), $R_{\mathcal{D}}(h) \leq \epsilon$

Figure 4: PAC

Realizability Assumption

- Assume there exists $h^* \in \mathcal{H}$ (class of functions) such that $R_{\mathcal{D}}(h^*) = 0$ (Implies h^* is the Bayes classifier)

Analysis of a Consistent Classifier

- For a **finite** hypothesis class $\mathcal{H}$, with probability $1 - \delta$:

$$R_{\mathcal{D}}(h) \leq \epsilon$$

- Sample complexity requirement:

$$m \geq \frac{\ln(|\mathcal{H}|/\delta)}{\epsilon}$$

(page 25-26-27 lecture 3 Learning Theory proof of this formula)

Feature	**Corollary 29: PAC Learnability**	**Corollary 31: Agnostic PAC Learnability**
Key Assumption	**Realizability Assumption**: Assumes there exists $h^* \in \mathcal{H}$ such that the true risk is zero ($R_{\mathcal{D}}(h^*) = 0$).	**Agnostic Setting**: Waives the realizability assumption; seeks a predictor with error not much larger than the best possible in H.
Sample Complexity	$m_{\mathcal{H}}(\epsilon, \delta) \leq \frac{\ln(\lvert\mathcal{H}\rvert/\delta)}{\epsilon}$	$m_{\mathcal{H}}(\epsilon, \delta) \leq \frac{\ln(2\lvert\mathcal{H}\rvert/\delta)}{2\epsilon^2}$
Learning Rule	Obtained by using the Empirical Risk Minimization (**ERM**) rule over a finite H.	Obtained by using the **ERM** rule over a finite H with a loss function bounded between 0 and 1.

Table 2: Comparison between standard PAC and Agnostic PAC learning.

Shattering & VC-dimension

- **Shattering:** $\mathcal{H}$ shatters S if all $2^{|S|}$ possible classifications are achievable using concepts in $\mathcal{H}$.
  - Let $|S| = n$. There are 2^n possible ways to assign binary labels (± 1) to these points.
  - **Intuition:** If $\mathcal{H}$ shatters S, it means that no matter what labels we assign to the points in S, we can always find a hypothesis $h \in \mathcal{H}$ that classifies them perfectly.
- **VC-dimension:** The cardinality of the largest set S that can be shattered by $\mathcal{H}$ The **Vapnik-Chervonenkis (VC) Dimension** is a measure of the capacity (complexity) of a hypothesis class.

$$\text{VCdim}(\mathcal{H}) = \max\{|S| : \mathcal{H} \text{ shatters } S\} \tag{1}$$

- If $\mathcal{H}$ can shatter sets of arbitrarily large size, then $\text{VCdim}(\mathcal{H}) = \infty$.
- To prove $\text{VCdim}(\mathcal{H}) = d$:
  1. Show there exists **at least one** set of size d that $\mathcal{H}$ can shatter.
  2. Show that **no** set of size $d+1$ can be shattered by $\mathcal{H}$.

- If infinite sets can be shattered, $VCdim(\mathcal{H}) = \infty$

While finite classes use cardinality $|\mathcal{H}|$ to determine learnability, infinite classes require VC-dimension to assess if they are PAC learnable.

Key Examples from the Slides For different geometric classifiers:

- **Thresholds on the real line**: (page 34) The VC-dimension is 1 because it can shatter one point but fails to shatter two points. For instance, it cannot label two points as positive if they require separate threshold boundaries.
- **Intervals on the real line**: These have a higher complexity than simple thresholds.
- **Linear Classifiers in 2D**: The VC-dimension is 3. While it can shatter any set of three non-collinear points, it cannot shatter all configurations of four points, such as when one point is inside the triangle formed by others or in XOR-like diagonal labelings.
- **Linear Classifiers in d dimensions**: The VC-dimension is $d+1$ (for $d \geq 2$).

Not PAC Learnable

Theorem: Let $\mathcal{H}$ be a class of infinite VC-dimension. Then $\mathcal{H}$ is not PAC learnable.

Intuitive Proof:

- If $\mathcal{H}$ shatters some set S of size $2m$, then we cannot learn $\mathcal{H}$ using m examples.
- Indeed, the first m samples give us no information about the labels of the rest of the instances in S.
- Every possible labeling of the rest of the instances can still be explained by some of the hypotheses in $\mathcal{H}$.

A theory that can explain everything and its contrary is worthless.

Discretization Trick

- In practice, algorithms use floating point (e.g., 32 bits per parameter)
- This makes the hypothesis class actually finite!
- **Rule of thumb:** Take a number of samples at least 10 times the number of parameters

In mathematical theory, many hypothesis classes $\mathcal{H}$ (like linear classifiers) are infinite because their parameters are real numbers $\mathbb{R}$. However, in practical application:

- **Computer Representation:** Any learning algorithm is implemented on a computer.
- **Floating Point Limits:** Parameters (coefficients) are stored in finite floating-point representations (e.g., 32-bit or 64-bit).
- **Finite Class:** Because there is a limited number of bit combinations, the class $\mathcal{H}$ is technically **finite**.

Mathematical Derivation

Suppose we have a model with d parameters, and each parameter is represented by 32 bits.

1. The number of possible values for one parameter is 2^{32}.
2. The total number of possible hypotheses is:

$$|\mathcal{H}_{discrete}| \leq (2^{32})^d = 2^{32d} \tag{2}$$

3. To find the sample complexity, we look at the natural log:

$$\ln |\mathcal{H}_{discrete}| \leq \ln(2^{32d}) = 32d \cdot \ln(2) \approx 22.18d \tag{3}$$

4. Using a conservative estimate for simplicity:

$$\ln |\mathcal{H}_{discrete}| \leq 10d \tag{4}$$

The 10x Rule of Thumb This discretization leads to a famous practical heuristic in Machine Learning:

> **Rule:** To achieve good generalization, the number of training samples m should be at least **10 times** the number of parameters d in the model.

Underfitting and Overfitting

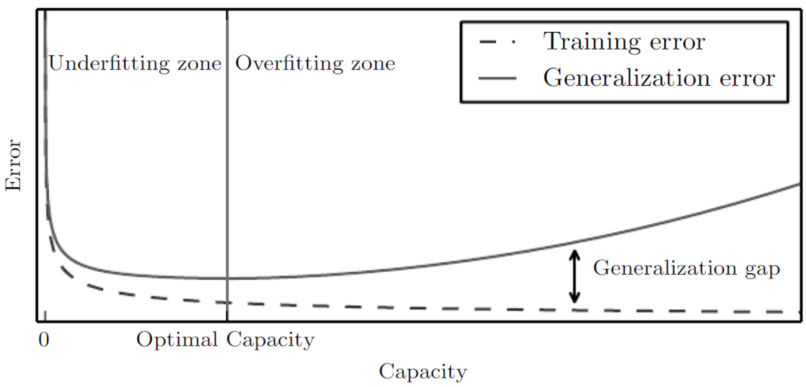


Figure 5: Things can go wrong

- **Underfitting zone:** High training and generalization error
- **Overfitting zone:** Low training error but high generalization gap
- **Optimal Capacity:** The point where generalization error is minimized

Underfitting vs. Overfitting

The lecture illustrates how model complexity (capacity) affects performance using polynomial regression $y(x, w) = \sum_{j=0}^{M} w_j x^j$ to fit a sine wave $t = \sin(2\pi x) + \epsilon$:

- **Underfitting**: Occurs when the model is too simple to capture the underlying data pattern. For example, using a polynomial of degree $M = 0$ or $M = 1$ results in a flat or straight-line prediction that misses the curves entirely.
- **Overfitting**: Occurs when a predictor performs excellently on the training set but poorly on the true "world". A high-degree polynomial (e.g., $M = 9$) can achieve zero training error by passing through every point, but it creates erratic oscillations that do not represent the true function.

The Generalization Gap

The **Generalization gap** is the difference between the Training error and the Generalization (true) error.

- As model capacity increases, training error typically decreases.
- Generalization error decreases only until the **Optimal Capacity** is reached; beyond this point, it increases as the model enters the **Overfitting zone**.

Why Overfitting Happens

Overfitting is not necessarily caused by noise. It stems from the fact that training error and true error are distinct measurements. It occurs when model complexity is too high relative to the number of available samples.

How to Prevent Things from Going Wrong

To ensure a model generalizes well, the following strategies are recommended:

- **Cross-Validation**: Partitioning data into Training, Validation, and Test sets. This allows for parameter tuning on the validation set without "peeking" at the test set, which must be used only for the final error estimate. **Cross-Validation: General vs. K-Fold**:
  - Cross-Validation (CV) is the general principle of using a subset of data for validation to prevent biased performance estimates.
  - K-Fold Cross-Validation This is the most standard implementation of CV:
    - **Algorithm:** The dataset S is divided into k folds: $S_1, S_2, \ldots, S_k$.
    - **Iteration:** For each $i \in \{1, \ldots, k\}$, train on $S \setminus S_i$ and test on S_i.
    - **Estimation:** The total error is the average of individual fold errors: $E_{total} = \frac{1}{k} \sum E_i$.
  - Comparison While "Cross-Validation" is the concept, **K-Fold** is the specific procedure used to minimize the variance of the error estimate compared to a simple holdout split.
- **Regularization**: Preferring shorter explanations (simpler models). This is achieved by adding a complexity penalty to the Empirical Risk Minimization (ERM) objective:

$$\mathrm{h}_S = \arg\min_{h \in \mathcal{H}} [R_S(h) + \lambda \mathrm{Complexity}(h)]. \quad (5)$$

- **Model Selection**: Finding the trade-off between model complexity and variance to approximate the Bayes function without fitting random fluctuations.

Train - Validation - Test

- **Training set:** Apply algorithm with different parameters
- **Validation set:** Choose the best h^* from $\mathcal{H}$
- **Test set:** Estimate the true error of h^*
- **CRITICAL:** Never use the test set for parameter selection!

Subset	Main Role	Optimization/Tuning	Bias/Estimation
Training Set	Used to produce a set of candidate hypotheses $H = \{h_1, \ldots, h_r\}$.	Minimizes **Empirical Risk** by optimizing internal weights.	Poor estimate of true risk due to potential overfitting.
Validation Set	Used for **Model Selection** to choose the best h^* from H.	Used for **Hyperparameter Tuning** to find the *Optimal Capacity.*	**Negatively biased**; the model was chosen because it fit this data.
Test Set	Used solely to estimate the final performance of the chosen model.	**NEVER** used for selection; must remain completely independent.	Provides an **unbiased estimate** of true generalization error.

Table 3: Comparison of Dataset Roles.

Linear Regression

Standard Loss Functions

- **Classification (0/1 Loss):** $L_{0/1}(\hat{y}, y) = \begin{cases} 0 & \hat{y} = y \\ 1 & \text{otherwise} \end{cases}$
- **Regression (Squared Loss):** $L(\hat{y}, y) = (\hat{y} - y)^2$

Ordinary Least Squares (OLS)

The objective is to minimize the **Squared Loss** between the actual labels y and the predicted values Xw.

- **Mathematical Objective:**
$$\min_w \|y - Xw\|_2^2$$
- **Gradient (Derivative):**
$$\nabla_w \|y - Xw\|_2^2 = 2X^\top(Xw - y)$$
- **Solution:** Setting the gradient to zero yields the closed-form solution:
  - Set to zero: $(X^\top X)w = X^\top y$
  - If $X^\top X$ is invertible:
$$w_{ols} := (X^\top X)^{-1} X^\top y$$

Ridge Regression(Regularized)

The objective adds a regularization term to penalize overly complex models (large weights), adhering to the principle of **Occam's Razor**.

- **Mathematical Objective:**
$$\min_w \frac{1}{2}\|y - Xw\|_2^2 + \frac{\alpha}{2}\|w\|_2^2$$
- **The Regularizer (α):** A positive parameter that controls the trade-off between fitting the training data and keeping the model simple to prevent **overfitting**.
- The regularizer α prevents the model from becoming too complex
- **Solution:**
$$w = (X^\top X + \alpha I)^{-1} X^\top y$$

Regression Type	Core Objective	Learning Goal	Solution
OLS	Minimize $\|y - Xw\|_2^2$	Minimize training error	$\mathrm{w}_{ols} = (X^\top X)^{-1} X^\top y$
Ridge	Minimize Loss + $\frac{\alpha}{2}\|w\|_2^2$	Control overfitting.	$\mathrm{w} = (\mathrm{X}^\top X + \alpha I)^{-1} X^\top y$

Table 4: Comparison of Regression Objectives.

Polynomial and Non-linear Regression

Polynomial regression allows us to fit non-linear data while keeping the optimization problem linear with respect to the weights w.

- **Basis Function Transformation:** We map the input x to a higher-dimensional space using a vector $\phi(x)$:
$$\phi(x) = [1, x, x^2, \ldots, x^M] \in \mathbb{R}^{M+1}$$
- **Predictor Form:** The resulting function is a linear combination of these basis functions:
$$\hat{y} = h(x) = \sum_{j=0}^{M} w_j x^j = \langle \phi(x), w \rangle$$
- **Generalization:** Other basis functions, such as **Radial Basis Functions (RBF)**, can also be used:
$$\phi_i(x) = \exp\left(-\frac{\|x - \mu_i\|^2}{s_i^2}\right)$$
- **Objective:** The optimal weights are found by minimizing the squared loss on the design matrix Φ:
$$\min_w \|y - \Phi w\|_2^2$$

Learning Theory Summary

- Learning is PAC (Probably Approximately Correct)
- ERM provides generalization guarantees under certain conditions
- Cross-validation is essential for model selection
- Regularization (e.g., Ridge) controls overfitting

4 Lecture 4 : Unsupervised Learning

Definition: A type of machine learning where the algorithm is trained on data without explicit labels or a "teacher."

4.1 Clustering

The process of grouping a set of objects into classes of similar objects.

- **High Intra-class Similarity:** Objects within the same cluster should be highly similar to one another.
- **Low Inter-class Similarity:** Objects in different clusters should be as distinct as possible.

4.2 The K-means Clustering Problem

Given a set of observations $(x_1, x_2, \ldots, x_n)$, where $x_i \in \mathbb{R}^d$.

4.2.1 Problem Definition

K-means clustering problem: Partition the n observations into K sets $(K \leq n)$ $\mathbf{S} = \{S_1, S_2, \ldots, S_K\}$ such that the sets minimize the within-cluster sum of squares:

$$\arg\min_{\mathbf{S}} \sum_{i=1}^{K} \sum_{x_j \in S_i} \|x_j - \mu_i\|^2 \tag{6}$$

where μ_i is the mean of points in set S_i.

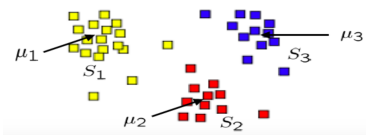


Figure 6: K means

4.2.2 Computational Complexity

The problem is **NP-hard**, but there are good heuristic algorithms that seem to work well in practice:

- K-means algorithm
- Mixture of Gaussians

4.2.3 Voronoi Diagrams in K-means

A **Voronoi Diagram** is the geometric representation of how space is partitioned in the K-means algorithm.

- **Definition:** For a set of centroids $\{\mu_1, \ldots, \mu_K\}$, the Voronoi cell V_i for centroid μ_i is defined as:

$$V_i = \{x \in \mathbb{R}^d : \|x - \mu_i\| \leq \|x - \mu_j\| \text{ for all } j \neq i\} \tag{7}$$

- **Geometric Properties:**
  - The boundaries between cells are **linear** (hyperplanes).
  - Each cell is a **convex polygon**.
- **Application:** K-means uses these diagrams to satisfy the **low inter-class similarity** goal by creating (iteratively) clear, distinct boundaries between clusters.

K-means Clustering Algorithm

Algorithm

Input

- Data + Desired number of clusters, K

Initialize

- the K cluster centers (randomly if necessary)

Iterate

1. Decide the class memberships of the n objects by assigning them to the nearest cluster centers.
2. Re-estimate the K cluster centers (aka the centroid or mean), by assuming the memberships found above are correct.

Termination

- If none of the n objects changed membership in the last iteration, exit.
- Otherwise go to 1.

- **At each iteration:**
  - Computing **distance** between each of the n objects and the K cluster centers is $O(Kn)$.
  - Computing **cluster centers**: Each object gets added once to some cluster: $O(n)$.
- **Total Complexity:** Assume these two steps are each done once for ℓ iterations: $O(\ell Kn)$.
  K-means is guaranteed to terminate in a finite number of iterations:
  - Each step reduces the within-cluster sum of squares.
  - There are a finite number of possible cluster partitionings.

  However, it is **not guaranteed** to find the global optimum, only a local one.

4.2.4 Challenges: Initialization and Local Minima

The results of the **K-means Algorithm** can vary significantly based on the initial **random seed selection**.

Key Observations

- **Convergence Quality:** Some seeds can result in a poor convergence rate or convergence to **sub-optimal clustering**.
- **Local Minima:** The K-means algorithm can get stuck easily in local minima.

Strategies for Better Clustering

To mitigate these issues, the following practices are recommended:

- **Heuristic Seed Selection:** Select good seeds using a heuristic, such as choosing an object that is least similar to any existing mean (e.g., object least similar to any existing mean).
- **Multiple Restarts:** Try out **multiple starting points**—this is considered very important for finding a stable solution.
- **Refinement:** Initialize the algorithm with the results of another clustering method.

The Logic of "Least Similar"

The goal of this heuristic is to spread the initial centers out as much as possible across your data.

Step 1: You pick the first center μ_1 randomly from the dataset.

Step 2: For the next center, you don't pick randomly. Instead, you calculate the distance from every data point to μ_1.

Step 3: You choose the next center μ_2 from the points that are **farthest away** (the "least similar") from μ_1.

Repeat: You keep doing this until you have all K centers.

4.2.5 What is K-means optimizing?

Define the following potential function F of centers μ and point allocation C (cluster assignment):

- $\mu = (\mu_1, \ldots, \mu_K)$
- $C = (C(1), \ldots, C(n))$

> **Tip: Allocation vs. Classification** While the vector C acts like a class label, there is a fundamental difference:
>
> - **Classification (Supervised):** Points are assigned to predefined *classes* based on known labels.
> - **Clustering (Unsupervised):** Points are *allocated* to clusters based on geometric similarity to a center μ. The "class" is discovered, not provided.

Two Equivalent Versions of the Potential Function

$$F(\mu, C) = \sum_{j=1}^{n} \|\mu_{C(j)} - x_j\|^2 \quad = \sum_{i=1}^{K} \sum_{j:C(j)=i} \|\mu_i - x_j\|^2 \tag{8}$$

Optimal solution of the K-means problem:

$$\min_{\mu, C} F(\mu, C)$$

4.2.6 K-means as Coordinate Descent

The K-means algorithm optimizes the potential function by alternating between two steps. This is a form of **Coordinate Descent on** F.

(1) Fix μ

$$\min_{C(1),\ldots,C(n)} \sum_{j=1}^{n} \|\mu_{C(j)} - x_j\|^2 = \sum_{j=1}^{n} \min_{C(j)} \|\mu_{C(j)} - x_j\|^2$$

Exactly first step: Assign each point to the nearest cluster center. This is the **Expectation step**.

(2) Fix C

$$\min_{\mu_1,\ldots,\mu_K} \sum_{i=1}^{K} \sum_{j:C(j)=i} \|\mu_i - x_j\|^2 = \sum_{i=1}^{K} \min_{\mu_i} \sum_{j:C(j)=i} \|\mu_i - x_j\|^2$$

Exactly 2nd step (re-center): Update the cluster centers to be the mean of assigned points. This is the **Maximization step**.

4.3 Gaussian Mixture Mode

4.3.1 Density Estimation: Generative Approach

In a generative approach, we assume the data is generated by some underlying probability distribution.

Model Assumptions

- There is a **latent parameter** Θ, the parameter of a probability distribution.
- For all i, we draw observed data x_i given Θ.
- **i.i.d. data:** We assume the data points are independent and identically distributed.

The likelihood of the data given the parameter is defined as:

$$p(x_1, \ldots, x_n|\theta) = \prod_{i=1}^{n} p(x_i|\theta) \tag{9}$$

> **Tip: What if the basic model doesn't fit all data?**
> If a single distribution cannot represent the data, we use **Mixture modelling** or **Partitioning algorithms**. This involves using different parameters $[\theta_1, \ldots, \theta_K]$ for different parts of the domain.

4.3.2 Partitioning Algorithms

Partitioning algorithms divide the data into groups using different assignment strategies.

1. K-means

- **Hard assignment:** Each object belongs to **only one** cluster.
- The parameter for an object i is chosen from a discrete set: $\theta_i \in \{\theta_1, \ldots, \theta_K\}$.

2. Mixture Modeling

- **Soft assignment:** An object has a **probability** of belonging to each cluster.
- This is represented by a vector of weights $(\pi_1, \ldots, \pi_K)$ where: $\pi_i \geq 0, \quad \sum_{i=1}^{K} \pi_i = 1$

4.4 Gaussian Mixture Models (GMM)

A Gaussian Mixture Model (GMM) is a **multi-modal distribution** where the data is assumed to be generated from K distinct Gaussian components.

4.4.1 The Generative Process

Each data point x is created using a two-step hierarchical process:

1. **Choose a Component:** Select a specific component $i \in \{1, \ldots, K\}$ K different "machines" (Gaussians) with probability $\pi_i = P(y = i)$. This π_i is called the *mixture proportion.*
2. **Generate the Point:** Sample the data point x from the chosen Gaussian distribution $N(\mu_i, \Sigma_i)$.

The total probability density of observing a point x is the sum of the probabilities from all possible components:

$$p(x) = \sum_{i=1}^{K} \underbrace{p(x|y = i)}_{\text{Mixture Component}} \underbrace{P(y = i)}_{\text{Mixture Proportion}} \tag{10}$$

Proportion: How likely is it that machine i was chosen?
Component: If machine i was chosen, how likely is it to produce exactly this point x?

4.4.2 Probabilistic Clustering: Mixture of Gaussians

In this framework, we assign a data point x to a cluster based on the posterior probability $P(y = i|x)$.

1. The Mathematical Setup

We assume that the following parameters of the mixture model are known:

- **Covariance:** $\Sigma_i = \sigma^2 \mathbf{I}$ for simplicity, implying all clusters are spherical and share the same size.
- **Component Density:** $p(x|y = i) = N(\mu_i, \sigma^2 \mathbf{I})$, which defines the Gaussian shape of cluster i.
- **Prior Probability:** $p(y = i) = \pi_i$, representing the proportion of the dataset belonging to cluster i.

2. Clustering based on Posteriors

To decide if a point x belongs to cluster i or cluster j, we calculate the **log-ratio** of their posterior probabilities:

$$\log \frac{P(y = i|x)}{P(y = j|x)} = \log \frac{p(x|y = i)P(y = i)/p(x)}{p(x|y = j)P(y = j)/p(x)} \tag{11}$$

By plugging in the Gaussian formula and simplifying (where second-order terms cancel out), we obtain a linear expression:

$$\log \frac{\pi_i \exp\left(\frac{-1}{2\sigma^2}\|x - \mu_i\|^2\right)}{\pi_j \exp\left(\frac{-1}{2\sigma^2}\|x - \mu_j\|^2\right)} = w^T x \tag{12}$$

The result $w^T x$ represents a **Linear Decision Boundary**.

the weight vector w used in the linear decision boundary $w^T x + c = 0$ is derived as:

$$w = \frac{1}{\sigma^2}(\mu_i - \mu_j) \tag{13}$$

This vector determines the classification of point x based on:

3. The Decision Rule

The sign of the log-ratio determines the final classification:

- **If** $\log \frac{P(y=i|x)}{P(y=j|x)} > 0$: x is assigned to **cluster** i.
- **If** $\log \frac{P(y=i|x)}{P(y=j|x)} < 0$: x is assigned to **cluster** j.
- **If** result $= 0$: x lies exactly on the **decision boundary**.

4.4.3 Maximum Likelihood Estimation (MLE)

When the parameters $\theta = \{\mu_1, \ldots, \mu_K, \sigma^2, \pi_1, \ldots, \pi_K\}$ are unknown, we use MLE to find the values that best explain our observed data. We do this by maximizing the log-likelihood:

$$\arg\max_{\theta} \sum_{j=1}^{n} \log \left(\sum_{i=1}^{K} P(y_j = i|\theta) P(x_j|y_j = i, \theta) \right) \tag{14}$$

> **Tip: The "Hidden" Variable**
> In this model, y (the identity of the component that created x) is a **hidden (latent) variable**. We see the points on the plot, but we do not know which specific Gaussian "die roll" produced them.

4.5 From K-means to General GMM

4.5.1 K-means as a Special Case of GMM

K-means can be derived from GMM by making two specific assumptions:

1. **Same Variance (σ^2):** Assume all components are isotropic (spherical) Gaussians with the same variance.
2. **Hard Assignment:** Assume the probability $P(y_j = i)$ is binary: 1 if the point belongs to cluster i, and 0 otherwise.

Under these constraints, maximizing the Likelihood (MLE) is mathematically equivalent to minimizing the K-means potential function $F(\mu, C)$:

$$\arg\max_{\theta} \prod_{j=1}^{n} P(x_j|\theta) = \arg\min_{\mu, C} \sum_{j=1}^{n} \|x_j - \mu_{C(j)}\|^2 = \arg\min_{\mu, C} F(\mu, C) \tag{15}$$

4.5.2 General GMM (Anisotropic Gaussians)

In a general GMM, we remove these restrictions. Components are **anisotropic**, meaning they can be elliptical and oriented in different directions.

- Each component i has its own mean μ_i and its own **covariance matrix** Σ_i.
- The total density remains: $p(x) = \sum_{i=1}^{K} \pi_i N(x; \mu_i, \Sigma_i)$.

4.5.3 Decision Boundaries

When we perform clustering based on posteriors (assigning x to the y that maximizes $P(y|x)$):

- **If Σ_i are the same:** The boundary is linear (like K-means Voronoi cells).
- **If Σ_i are different:** The second-order terms in the Gaussian formula do not cancel out, resulting in a **Quadratic Decision Boundary**.

The log-posterior ratio follows the form: $\log \frac{P(y=i|x)}{P(y=j|x)} = x^T W x + w^T x + c$.

4.5.4 Estimation: Why we use EM

To find the optimal parameters $\theta = [\mu_1, \ldots, \mu_K, \Sigma_1, \ldots, \Sigma_K, \pi_1, \ldots, \pi_K]$:

1. Setting the derivative of the log-likelihood to zero results in non-linear equations that are **not analytically solvable**.
2. **Gradient Descent** is possible but often slow.
3. **Expectation-Maximization (EM)** is the standard iterative approach used to solve this.

4.5.5 General GMM: Maximum Likelihood Estimation (MLE)

When the parameters $\theta = [\mu_1, \ldots, \mu_K, \Sigma_1, \ldots, \Sigma_K, \pi_1, \ldots, \pi_K]$ are unknown, we aim to find the values that maximize the marginal likelihood of our observed data.

The MLE Objective Function

The goal is to solve for θ using the following maximization:

$$\begin{aligned}
\arg\max_{\theta} \prod_{j=1}^{n} P(x_j|\theta) &= \arg\max_{\theta} \prod_{j=1}^{n} \sum_{i=1}^{K} P(y_j = i, x_j|\theta) \\
&= \arg\max_{\theta} \prod_{j=1}^{n} \sum_{i=1}^{K} P(y_j = i|\theta) p(x_j|y_j = i|\theta) \\
&= \arg\max_{\theta} \prod_{j=1}^{n} \sum_{i=1}^{K} \pi_i \frac{1}{\sqrt{2\pi|\Sigma_i|}} \exp\left[-\frac{1}{2}(x_j - \mu_i)^T \Sigma_i^{-1} (x_j - \mu_i)\right]
\end{aligned}$$

How to solve for θ

Finding the specific θ that maximizes this likelihood presents several challenges:

- **Analytical Solution:** Setting the derivative of the log-probability to zero ($\frac{\partial}{\partial \mu_i} \log \text{Prob}(\ldots) = 0$) results in non-linear equations. These are **non-analytically solvable**.
- **Gradient Descent:** This method is doable but often inefficient and **slow** for these specific types of problems.
- **EM Algorithm:** The preferred approach is to use the **Expectation-Maximization (EM)** algorithm to iteratively estimate the parameters.

4.5.6 Comparison of Models

Feature	**Basic GMM (Isotropic)**	**General GMM (Anisotropic)**
Geometry	Perfect Spheres	Ellipses / Ovals
Variance (Σ)	Shared $\sigma^2\mathbf{I}$	Unique Σ_i for each cluster
Decision Boundary	Linear (Straight lines)	Quadratic (Curved lines)
Complexity	Lower (fewer parameters)	Higher (more parameters to estimate)

> **Tip:** **Summary:** While the General GMM is more complex to estimate, it is much more powerful because it can model data where clusters have different sizes, orientations, and densities.

4.6 Introduction to EM

EM is a general algorithm to deal with **hidden data**. In the context of unsupervised learning, these hidden values are the cluster class labels.

- **Optimization Strategy:** EM optimizes objective functions interpreted as likelihoods in the presence of missing data.
- **Simplicity:** Unlike gradient methods, there is **no need to choose a step size**.
- **Iterative Nature:** Consists of two linked steps:
  - **E-step:** Fill-in hidden values using inference.
  - **M-step:** Apply standard MLE/MAP methods to the "completed" data.
- **Convergence:** The procedure **monotonically improves** the likelihood and always converges to a local optimum.

4.6.1 Simple vs. General GMMs

See details in pageds 38-39-40.. lecture 4(1))

The Simple Case (Spherical)

In this restricted scenario, all clusters are assumed to be perfect spheres of the same size. The algorithm only needs to learn the location (means) of the clusters. This is essentially a "soft" version of the K-means algorithm.

The General Case (Anisotropic)

In the general case, we remove these restrictions. The algorithm must simultaneously learn:

- **Locations:** The mean of each cluster.
- **Shapes:** The specific elliptical orientation and "stretch" (covariance) of each cluster.
- **Proportions:** The relative size or importance (priors) of each cluster in the total mixture.

Lecture 4bis: Supervised Learning Algorithms: KNN, Perceptron, Kernels, MLP

4.7 K-Nearest Neighbors (KNN) in Supervised Learning

The fundamental intuition of KNN is that similar points (those nearby in feature space) likely share the same label.

Comparison: KNN (Supervised) vs. Nearest Neighbors (Unsupervised)

The following table provides a mathematical and conceptual comparison between Supervised and Unsupervised neighbor-based approaches as discussed in the course materials.

Feature	KNN (Supervised Learning)	Nearest Neighbors (Unsupervised)
Primary Goal	Predict the class label or continuous value of an unseen query point.	Discover hidden structures, groupings, or manifolds in data .
Data Requirements	Requires labeled training set $\{(x_i, y_i)\}_{i=1}^n$.	Requires only input features $\{x_i\}_{i=1}^n$; no labels provided.
Fundamental Assumption	**Smoothness:** Nearby points in feature space likely share the same label.	**Cluster/Manifold:** Nearby points belong to the same local geometric structure.
Mathematical Basis	**Decision Function:** $f(x) = \text{mode}(y_{i1}, \ldots, y_{ik})$.	**Objective Function:** Often minimizes intra-cluster distance or preserves local geometry.
Distance Formulas	Uses L_p norms. For categorical data: $L_H(x, y) = \sum_{i=1}^d \mathbf{1}_{x_i \neq y_i}$.	Identical L_p norms used to define similarity matrices or cluster boundaries.
Constraints	Must store/search all N training points; $O(ND)$ time complexity.	Local searches may fail to capture global data shape if k is too small.
Hyper-parameters	k: Small k increases variance; large k increases bias.	K: Determines the granularity of the discovered groupings.

Table 5: Comparative analysis of neighbor-based learning paradigms

Practical Constraints & Preprocessing
Both methods are sensitive to the "Curse of Dimensionality" and require **Feature Normalization** (standardizing to mean 0 and variance 1) to ensure all features contribute equally to the distance calculation.

4.7.1 Distance Metrics

The choice of distance $D(X, X_i)$ defines the behavior of the algorithm. Common L_p norms include:

- **Manhattan (L_1)**: $\sum |x_i - y_i|$.
- **Euclidean (L_2)**: $\sqrt{\sum (x_i - y_i)^2}$.
- **Chebyshev (L_∞)**: $\max |x_i - y_i|$.
- **Hamming** (L_H):$L_H(x, y) = \sum_{i=1}^d \mathbf{1}_{x_i \neq y_i}$

4.7.2 kNN Pros and Cons

.

Pros	Cons
Minimal Data Assumptions: • Smoothness: nearby regions of space share the same class. • Distance function assumptions apply only locally.	**Computationally Expensive:** • Space: requires storing all training examples. • Time: distance computation for all examples is $O(ND)$.
Non-Parametric Approach: • "Let the data speak for itself". • Nothing to infer except k and the distance function. • Easy online updates by adding new items to the set. • Handles increasing numbers of classes without issues.	**Scalability Issues:** • As N (number of examples) grows, the system becomes slower. • D represents the cost of computing distance. • Computational burden is at **testing** time, not training (inefficient).
Generalization: • Good guarantees for large sample sizes. • Error rate is less than twice the Bayes error.	

Table 6: Pros and Cons KNN.

4.7.3 kNN Tricks (Reduce Complexity)

1. Partial Distance Calculation

To avoid full $O(D)$ computation for every neighbor, we calculate distance using a subset r of the full d dimensions. If D_r exceeds a threshold, we stop:

$$D_r(\mathbf{a}, \mathbf{b}) = \left(\sum_{k=1}^{r} (a_k - b_k)^2 \right)^{1/2} \quad \text{where } r < d$$

Assumption: The dimensional subspace r is indicative of the full data space.

2. Pre-structuring / Searching for Prototypes

Data is represented as search trees of prototypes to decrease search complexity.

- **Pros:** Significant reduction in time complexity.
- **Cons:** Finding a good tree is non-trivial; it may not guarantee finding the absolute nearest neighbor.

3. Editing (Data Pruning)

Prune "useless" points during training. A point is removed if all its k nearest neighbors share the same class.

- Keeps decision boundaries and Voronoi Tessellation intact while reducing N.
- **Constraint:** Prevents the addition of new training data later.

4. Separate Feature Normalization

Prevents features with large scales from dominating the distance metric. If X has mean μ and variance σ^2, the normalized feature is computed:

$$\hat{x}_i = \frac{x_i - \text{mean}(x_i)}{\sqrt{\text{var}(x_i)}}$$

This ensures all features have mean 0 and variance 1.

4.8 Non-linearity and Feature Maps

Many real-world datasets are not linearly separable in their original input space $\mathcal{X}$. To address this, we apply a non-linear mapping to a higher-dimensional feature space $\mathcal{F}$:

$$\Phi : \mathcal{X} \to \mathcal{F}$$

In this new space, a linear decision boundary (hyperplane) can often separate the data effectively.

4.8.1 Example: Circular Separation

If data is distributed in a circle, it is impossible to separate with a line $w^T x + b = 0$. However, by mapping the 2D input to a 3D space using quadratic features, the data becomes linearly separable by a plane in the transformed space.

4.9 Computational Efficiency Problem

Extracting high-order features manually is extremely costly:

- **High Dimensionality:** Using second-order polynomial features in a 1,000-dimensional space results in approximately $5 \cdot 10^5$ dimensions.
- **Scaling Issues:** For higher-order polynomials or infinite-dimensional maps, the number of features becomes computationally prohibitive to store or process.

4.10 The Solution: Implicit Computation

Instead of computing $\Phi(x)$ explicitly, we use a **Kernel Function** $k(x, x') = \langle \Phi(x), \Phi(x') \rangle$.

- If $k(x, x')$ is cheaper to compute than $\Phi(x)$, we can solve high-dimensional problems efficiently without ever "visiting" the high-dimensional space.

4.11 Feature Engineering and Nonlinear Mappings

4.11.1 The Core Concept

The goal of feature engineering is to transform input data $x \to \Phi(x)$ so that the problem becomes solvable by linear classifiers.

- **Prior Knowledge:** It allows the encoding of domain expertise into the model.
- **Nonlinear Relationships:** It captures relationships that a simple linear model would miss.

4.11.2 Practical Applications

Spam Filtering

A raw email is transformed into features such as:

- **Bag of words:** Presence/absence of specific keywords.
- **Pairs of words:** Analyzing associations between words.
- **Metadata:** Date, time, IP address, and sender authentication.
- **Technical features:** Recipient path, encoding types, and links.

Complex Geometries

- **Two Interlocking Spirals:** Solved by transforming Cartesian coordinates (x_1, x_2) into radial and angular parts $(r \sin \phi, r \cos \phi)$.
- **Circular Separation:** Solved via quadratic features (e.g., x_1^2, x_2^2).

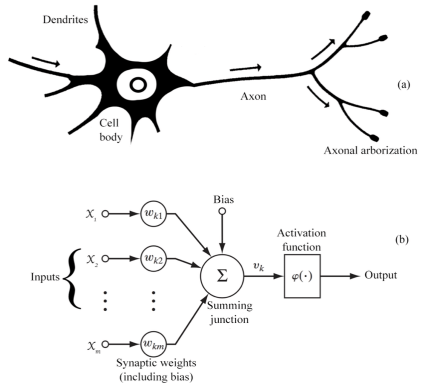
Other Domains

- **Medical Diagnosis:** Combining physician comments, blood status, ECG, and biological measurements (height, weight, temperature).
- **Character Recognition:** Breaking down images of Japanese characters into specific strokes and lookup tables based on stroke order .

5 Lecture 5: Multi-Layer Perceptrons and Optimization

5.1 Artificial Neurons and the Perceptron

The artificial neuron (perceptron) is inspired by biological neurons, where synapses are represented by weights and the soma by a linear combiner followed by an activation function.



Functional Architecture

The Perceptron is a linear classifier defined by the following mathematical components:

- **Weighted Linear Combination:** Computes the inner product of input features and synaptic weights: $\langle w, x \rangle = \sum_i w_i x_i$.
- **Linear Offset (Bias):** A bias term b is added to shift the decision boundary from the origin.
- **Nonlinear Decision Function:** The final output is determined by a nonlinearity σ:

$$f(x) = \sigma(\langle w, x \rangle + b) \tag{16}$$

Geometric Interpretation

The Perceptron defines a **linear separating hyperplane** in the feature space.

- It classifies data by determining which side of the hyperplane a point falls on .
- Typical applications include binary tasks such as spam vs. ham filtering or novel vs. typical data detection.

Learning Algorithm

Learning is the process of estimating parameters w and b using an iterative approach:

1. Initialize $w = 0$ and $b = 0$
2. Pick a training instance (x_i, y_i) where $y \in \{-1, +1\}$.
3. **Update Rule:** If the point is misclassified ($y_i[\langle w, x_i \rangle + b] \leq 0$):
   - $w \leftarrow w + y_i x_i$
   - $b \leftarrow b + y_i$
4. Repeat until all points are classified correctly.

Convergence and Hardness

- **Convergence Theorem:** If a linear separator (w^*, b^*) with unit length exists such that $y_i[\langle x_i, w^* \rangle + b^*] \geq \epsilon$, the algorithm converges.
- **Bound on Steps:** The maximum number of updates is bounded by $(b^{*2} + 1)(r^2 + 1)\rho^{-2}$, where $\|x_i\| \leq r$.
- **Properties:** Convergence is dimensionality-independent and order-independent, scaling instead with the *difficulty* (margin vs. size) of the problem.
- **Sparsity:** The final weight vector $w = \sum_{i \in I} y_i x_i$ is a linear combination of only the instances that caused errors, providing a **compression bound**.
- **Limitations:** Standard Perceptrons fail with noisy data and cannot solve non-linearly separable problems like **XOR**. Finding the minimum error separator for such cases is **NP-hard**.

5.1.1 Mathematical Model

A single neuron (Perceptron) computes a weighted sum of its inputs x_i and applies an activation function σ:

$$net = \sum_{i=0}^{n} w_i x_i \quad \text{where } x_0 = 1 \text{ is the bias term} \tag{17}$$

The output is given by $o = \sigma(net)$. Common activation functions include:

- **Heaviside Step Function:** $Y = +1$ if $X \geq \omega_0$, else -1.
- **Sigmoid Function:** $\sigma(net) = \frac{1}{1+e^{-net}}$.
- **Sigmoid Derivative Property:** $\frac{d\sigma}{d(net)} = \sigma(1 - \sigma)$.

The goal of the learning algorithm is to find a weight vector $\vec{w}$ that minimizes the error between the network's output and the target values.

5.2 Error Function: Mean Squared Error (MSE)

We define the error E over the set of training examples D as:

$$E[\vec{w}] \equiv \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \tag{18}$$

where t_d is the target output and o_d is the observed output for example d.

5.3 Gradient Descent Intuition

To minimize E, we calculate the gradient $\nabla E[\vec{w}]$, which points in the direction of the steepest increase in error. We move in the **opposite** direction:

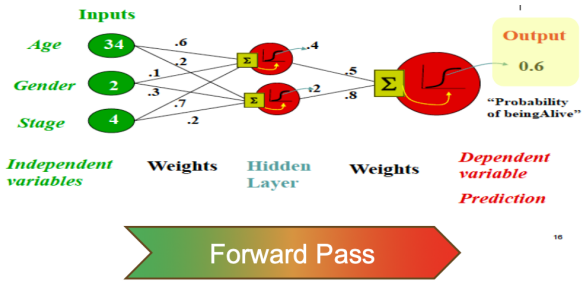
- **The Gradient Vector:** $\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \ldots, \frac{\partial E}{\partial w_n}\right]$.
- **The Training Rule:** $\Delta w_i = -\eta \frac{\partial E}{\partial w_i}$.
- **Learning Rate (η):** A constant that determines the step size toward the minimum.

5.4 Incremental (Stochastic) Gradient Descent

In practice, we often update the weights after every training example rather than waiting for the whole batch. For a sigmoid unit, the update rule for each weight w_i is:

$$w_i \leftarrow w_i + \eta(t_d - o_d)o_d(1 - o_d)x_{i,d} \tag{19}$$

where $(t_d - o_d)$ is the error and $o_d(1 - o_d)$ is the derivative of the sigmoid activation function.



5.5 Limitations: The XOR Problem

A single perceptron defines a **linear decision surface**.

- It can successfully learn linearly separable gates (AND, OR, NAND).
- It **cannot** solve the XOR gate, which requires hidden layers and the Backpropagation algorithm to learn non-linear surfaces.

5.6 Multi-Layer Perceptron (MLP)

An MLP is a network of neurons organized in layers: an **Input Layer**, one or more **Hidden Layers**, and an **Output Layer**.

- MLPs can learn highly non-linear functions, such as the XOR gate, which a single-layer perceptron cannot solve.
- **Forward Pass:** The process of computing outputs by passing input signals through successive layers.

5.7 Training and Backpropagation

Training involves minimizing an error function, typically the **Mean Squared Error (MSE)**:

$$E[\vec{w}] = \frac{1}{2} \sum_{d \in D} (t_d - o_d)^2 \tag{20}$$

where t_d is the target and o_d is the observed output.

5.7.1 Backpropagation Algorithm

Backpropagation uses the chain rule to calculate the gradient of the error with respect to every weight in the network.

1. **Output Unit Error:** $\delta_k = o_k(1 - o_k)(t_k - o_k)$.
2. **Hidden Unit Error:** $\delta_h = o_h(1 - o_h) \sum_{k \in outputs} w_{hk} \delta_k$.
3. **Weight Update:** $\Delta w_{ij} = \eta \delta_j x_i$, where η is the learning rate.

5.8 Hierarchical Representations in Deep Learning

Deep Learning is defined as the process of learning a hierarchy of internal representations. The network architecture typically consists of multiple stages of **Trainable Feature Extractors** followed by a **Trainable Classifier**.

Biological Inspiration The architecture of modern ANNs is inspired by the human visual system's processing path :

- Information travels from the **Retina** (20-40 ms) through **V2** and **V4** to the **VT** (100-130 ms).
- Simple visual forms (edges/corners) are processed first, followed by intermediate forms, and finally high-level descriptions for decision making in the **PFC** .

Domain Examples

Hierarchy Level	Vision Hierarchy	Language Hierarchy
Low	Pixels → Edges	Words → Parts of Speech
Mid	Textons → Parts	Phrases → Statements
High	Objects → Scenes	Text → Stories

Table 7: Hierarchical representations across different data domains .

Why Hierarchies? Using multiple stages allows the network to learn representations that are **increasingly invariant** to noise, rotation, or scale as the information moves up the layers, enabling the final classifier to make accurate categorical judgments.

5.9 Optimization Methods

In optimization, the choice of loss function L defines the shape of the error surface. For linear functions of weights $f(x) = w^T x$, we use the following:

Loss Type	Mathematical Expression	Typical Application
Squared Loss	$L = (y_i - f(x_i))^2$	Regression tasks
Hinge Loss	$L = \max(0, 1 - y_i f(x_i))$	Classification ($y \in \{-1, 1\}$)
Cross-Entropy	$L = \log(1 + e^{-y_i f(x_i)})$	Logistic Regression

- **Scaling**: RMSprop and ADAGRAD are particularly useful for scaling these gradients when different parameters have wildly different scales.

5.9.1 Stochastic Gradient Descent (SGD)

Instead of using the full dataset (Batch), SGD uses a **Minibatch** to compute an unbiased estimate of the gradient $g^{(t)}$, allowing for faster updates.

The Saddle Point Problem

In deep networks, the primary obstacle is often not local minima but **saddle points**. At these points, the gradient $\nabla_W L$ vanishes, leading to:

- **Slow Progress**: Gradients are very small in the vicinity of the saddle, making updates nearly zero.
- **Prevalence**: Evidence suggests that in high-dimensional non-convex optimization, zeros of the gradient are most likely to be saddles.

5.9.2 Advanced Optimizers

SGD, SGD+Momentum, Adagrad, RMSProp, Adam all have learning rate as a hyperparameter.

- **Momentum:** Accelerates SGD by navigating the relevant directions and softening oscillations: $\Delta w_{i,j}(t) = \eta \delta_j x_{i,j} + \alpha \Delta w_{i,j}(t-1)$.

- **ADAGRAD/RMSprop:** Diagonally scale gradients, useful for datasets with varying gradient magnitudes (e.g., text).
- **ADAM:** Combines momentum and RMSprop to scale the gradient and apply moving averages.

RMSprop vs. ADAGRAD

Both optimizers scale the gradient to handle varying magnitudes, but they differ in their decay:

- **ADAGRAD**: Uses the *cumulative* sum of squared gradients. The learning rate decays at $1/\sqrt{t}$, which can be too aggressive for long training.
- **RMSprop**: Uses a *moving average* (controlled by β). This keeps the learning rate fixed across time, making it better for complex, long-running tasks.

Weight Momentum

Accelerates SGD by navigating the relevant directions and softening oscillations; to bypass small gradients and navigate flat regions of the loss landscape, we add a momentum term α:

$$\Delta w_{i,j}(t) = \eta \delta_j x_{i,j} + \alpha \Delta w_{i,j}(t-1) \tag{21}$$

This allows the update to "persist" in its state of motion, effectively acting as a physical viscosity.

5.10 Optimization Summary

- **Learning Rate Scheduling** Selecting a fixed learning rate α is difficult. Scheduling α to decay over time often leads to better convergence:
  - **Exponential Decay**: $\alpha = \alpha_0 e^{-kt}$
  - **Step Decay**: Reducing α by a factor every N epochs
  - **1/t Decay**: $\alpha = \alpha_0/(1+kt)$
- **Regularization (Weight Decay)** L_2 regularization allows for a controlled **Bias-Variance Tradeoff**.

$$L_{total} = L_{data} + \lambda \sum w_{ij}^2 \tag{22}$$

  - Strong Regularization: Lowers variance, increases bias; prevents fitting noise .
  - Validation Impact: Proper tuning of the weight decay coefficient is essential for maximizing validation accuracy .
- **Local Minima:** While they exist, in high-dimensional complex networks, most local minima are clustered near the global minimum, making them less of a problem than previously thought.
- **Local Minima vs. Saddle Points** Contrary to early AI fears, local minima are usually not a major problem for complex ANNs:
  - Similarity: Most local minima achieve a loss nearly as low as the global minimum.
  - The Saddle Point Caveat: Most points where $\nabla E = 0$ are **saddle points**, which slow training significantly due to vanishing gradients.

From Perceptrons to Deep Learning

Decision Surfaces and XOR

A single perceptron is limited to **linear separating hyperplanes**.

- **Realizability:** It can model NAND (not and) gates but fails on XOR because the latter is not linearly separable.
- **Non-Linearity:** Multi-stage processing is required to create non-linear decision boundaries.

Multi-Layer Perceptron (MLP) Model

The MLP approximates complex functions by *"Stacking More Layers"*:

- **Structure:** Input Layer → Hidden Layer(s) → Output Layer.
- **Combined Logistic Models:** The network functions as a sequence of logistic regressions where each layer provides inputs to the next.

Lecture 5_{bis}: Support Vector Machines (SVM)

5.11 The Core Concept: Large Margin Separation

The fundamental goal of an SVM is to find the **most robust** linear separator among all possible valid classifie. This is achieved by maximizing the **margin** ρ, which is the distance between the decision boundary and the closest training samples (Support Vectors).

5.11.1 Mathematical Formulation

For a linear function $f(x) = \langle w, x \rangle + b$, we define the boundaries such that:

- $\langle w, x \rangle + b \geq 1$ for positive instances.
- $\langle w, x \rangle + b \leq -1$ for negative instances.

The margin width is derived as $\frac{1}{\|w\|}$ (slide 18/84) . Maximizing the margin is equivalent to the following **Primal Optimization Problem**:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{subject to } y_i(\langle w, x_i \rangle + b) \geq 1, \forall i \tag{23}$$

We can solve it thanks to a mathematician from Turin!; Method of Lagrange multipliers.

5.12 The Dual Problem and Support Vectors

Using **Lagrange Multipliers** (α_i), the problem is transformed into a **Dual Problem** that only depends on the inner products of instances (slide 28/84):

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \quad \text{s.t.} \quad \sum \alpha_i y_i = 0, \alpha_i \geq 0 \tag{24}$$

5.12.1 Key Properties

- **Sparsity**: Only points on the margin (**Support Vectors**) have $\alpha_i > 0$. All other points can be ignored (Karush Kuhn Tucker Optimality condition; slide 39/84).
- **Weight Vector**: The optimal weights are a weighted linear combination of instances: $w = \sum_i y_i \alpha_i x_i$.

Properties of SVM Optimization (Slide 46) The Dual formulation of SVM reveals three critical properties:

- **Inner Product Dependency:** The optimization objective depends only on $\langle x_i, x_j \rangle$, which allows for the **Kernel Trick** ($k(x_i, x_j)$) to solve non-linear problems.
- **Sparsity:** The final decision function $f(x) = \sum_i \alpha_i y_i k(x_i, x) + b$ only involves **Support Vectors** (points where $\alpha_i > 0$); all other points are irrelevant.
- **Robustness:** By maximizing the margin $\frac{1}{\|w\|}$, the classifier ensures maximum distance from all training points, providing robustness against data uncertainty.

Why Large Margins? Maximum robustness relative to uncertainty, Independence of correctly classified instances, Easy to find for easy problems

5.13 Soft Margin and Slack Variables

When data is not linearly separable, we introduce **Slack Variables** (ξ_i) to allow some misclassifications (a convex optimization problem, that's always feasible by adding slack variables, meaning a solution always exists even if the data is messy :

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{s.t. } y_i(\langle w, x_i\rangle + b) \geq 1 - \xi_i, \xi_i \geq 0 \tag{25}$$

The Hyperparameter C: Controls the trade-off between maximizing the margin and minimizing classification errors.

- **Large** C: Prioritizes correct classification (narrower margin).
- **Small** C: Prioritizes a larger margin (more robust, allows more slack).

The Dual Problem (Slide 70) To solve the soft margin optimization problem, we construct the Lagrange function by incorporating the slack variables ξ_i and their respective multipliers. The resulting Dual Problem is derived by ensuring the derivatives with respect to w, b, and ξ vanish:

- **Dual Objective:** Maximize the function with respect to α:

$$\max_{\alpha} \left(\sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j\rangle \right) \tag{26}$$

- **Constraints:**
  * $\sum_i \alpha_i y_i = 0$
  * $0 \leq \alpha_i \leq C$ (The upper bound C limits the influence of any single instance)

Karush-Kuhn-Tucker (KKT) Conditions (Slide 73) The KKT conditions define the requirements for an optimal solution $(\vec{w}, b)$ and provide the mathematical definition of Support Vectors.

- **Stationarity:** $w = \sum_i y_i \alpha_i x_i$
- **Complementary Slackness:**
  * $\alpha_i[y_i(\langle w, x_i\rangle + b) + \xi_i - 1] = 0$
  * $\eta_i \xi_i = 0$
- **Point Classification based on α_i:**
  * $\alpha_i = 0 \implies y_i(\langle w, x_i\rangle + b) \geq 1$ (Point is correctly classified outside or on the margin)
  * $0 < \alpha_i < C \implies y_i(\langle w, x_i\rangle + b) = 1$ (Point is a **Support Vector** exactly on the margin)
  * $\alpha_i = C \implies y_i(\langle w, x_i\rangle + b) \leq 1$ (Point is a **Misclassified point** or inside the margin)

5.14 Nonlinear Separation: The Kernel Trick

To handle non-linear data, we map instances into a higher-dimensional space where they become linearly separable. We replace the inner product $\langle x, x'\rangle$ with a **Kernel Function** $k(x, x') = \langle \Phi(x), \Phi(x')\rangle$.

5.14.1 The Kernel Perceptron

The Kernel Perceptron allows learning in the feature space without explicit mapping:

The Kernel Perceptron Algorithm

Algorithm[cite: 1]

Input

- Training data pairs (x_i, y_i) and a valid kernel function $k(x, x')$.[cite: 1]

Initialize

- $f = 0$ (Initial decision function).[cite: 1]

Iterate

1. Pick a sample (x_i, y_i) from the data.[cite: 1]
2. Check the classification condition: if $y_i f(x_i) \leq 0$ (misclassified), update the function.[cite: 1]
3. **Update Rule**: $f(\cdot) \leftarrow f(\cdot) + y_i k(x_i, \cdot) + y_i$.[cite: 1]

Termination

- If $y_i f(x_i) > 0$ for all i (all points correctly classified), exit.[cite: 1]
- Otherwise, continue iterating through the data.[cite: 1]

- **Weight Vector Expansion:** The weight vector is a linear combination of the mapped instances: $w = \sum_{i \in I} y_i \Phi(x_i)$.
- **Non-linear Separator:** The classifier is a linear combination of inner products (kernels):

$$f(x) = \sum_{i \in I} y_i \langle \Phi(x_i), \Phi(x) \rangle + b = \sum_{i \in I} y_i k(x_i, x) + b$$

5.14.2 Common Kernel Functions

- **Linear Kernel**: $k(x, x') = \langle x, x' \rangle$.
- **Polynomial Kernel**: $k(x, x') = (\langle x, x' \rangle + c)^d$ where $c > 0$ and $d \in \mathbb{N}$.
  - **Proof Strategy**: Corresponds to a dot product because it can be expanded into an explicit sum: $\sum_{i=0}^{m} \binom{d}{i} (\langle x, x' \rangle)^i c^{d-i}$
- **RBF (Gaussian) Kernel**: $k(x, x') = \exp(-\gamma \|x - x'\|^2)$.

The **Gaussian Kernel Hyperparameter** (σ or γ):
$\sigma \to \infty$: Leads to **Underfitting**. $\sigma \to 0$: Leads to **Overfitting**.

5.14.3 Kernel Conditions and Mercer's Theorem

For a function $k(x, x')$ to be a valid kernel, it must satisfy specific conditions:

- **Computability**: It must be efficient to compute, ideally much cheaper than the dot products in the feature space itself.
- **Symmetry**: $k(x, x') = k(x', x)$ because $\langle \Phi(x), \Phi(x') \rangle = \langle \Phi(x'), \Phi(x) \rangle$.
- **Mercer's Theorem**: For any symmetric, square-integrable function k, if $\int \int k(x, x') f(x) f(x') dx dx' \geq 0$ for all $f \in L_2(\mathcal{X})$, then there exist ϕ_i and $\lambda_i \geq 0$ such that $k(x, x') = \sum_i \lambda_i \phi_i(x) \phi_i(x')$.

5.14.4 Mathematical Properties

- **Kernel Matrix (K)**: A matrix where $K_{ij} = k(x_i, x_j)$. It serves as a **Similarity Measure**, telling us the overlap between $\Phi(x_i)$ and $\Phi(x_j)$.
- **Positive Semidefiniteness**: All valid kernel matrices K must satisfy $\alpha^\top K\alpha \geq 0$ for all $\alpha \in \mathbb{R}^m$.
- **Distance in Feature Space**: The distance between points is calculated as:

$$d(x, x')^2 = k(x, x) + k(x', x') - 2k(x, x')$$

6 Lecture 6:

6.1 Logistic Regression for Classification

Linear regression is unsuitable for classification as it predicts values in $\mathbb{R}$ rather than probabilitie.

- **Sigmoid Function:** Maps linear outputs to probabilities between 0 and 1: $g(h) = \frac{1}{1+e^{-h}}$.
- **Probabilistic Model:**

$$p(y=1|X;w) = \frac{e^{w^T X}}{1+e^{w^T X}} \quad | \quad p(y=0|X;w) = \frac{1}{1+e^{w^T X}}$$

- **Optimization:** Unlike linear regression, there is no closed-form solution[cite: 5]. We use **Gradient Ascent** to find the Maximum Likelihood Estimate (MLE).
- **Likelihood of the Data:** The probability of observing the dataset given parameters w is defined as:

$$L(y|X;w) = \prod_i (1-g(X_i;w))^{y_i} g(X_i;w)^{(1-y_i)}$$

.

- **Log-Likelihood (LL):** Taking the natural log transforms the product into a sum for easier optimization[cite: 1]:

$$LL(y|X;w) = \sum_{i=1}^{N} y_i \ln \frac{1-g(X_i;w)}{g(X_i;w)} + \ln g(X_i;w)$$

.

- **Optimization:** Logistic Loss is also known as **Negative Log-Likelihood** ($\mathcal{L}(y, f(x)) = -LL(y|X;w)$). Since there is no closed-form solution, we use **Gradient Ascent** to find the Maximum Likelihood Estimate (MLE).
- **Regularization (MAP):** Incorporating a Gaussian prior on weights leads to L_2 regularization, preventing individual weights from becoming too large.

6.1.1 Maximum Likelihood Estimation (MLE)

To find the optimal weights w, we derive the log-likelihood function with respect to each component w^j of the weight vector:

- **Partial Derivative:** Taking the derivative w.r.t. w^j:

$$\begin{aligned}
\frac{\partial}{\partial w^j} l(w) &= \frac{\partial}{\partial w^j} \sum_{i=1}^{N} \{y_i w^T X_i - \ln(1+e^{w^T X_i})\} \\
&= \sum_{i=1}^{N} X_i^j \{y_i - (1-g(X_i;w))\} \\
&= \sum_{i=1}^{N} X_i^j \{y_i - p(y^i = 1|X_i;w)\}
\end{aligned}$$

- **Function Characteristics:**
  - **Bad news:** There is no closed-form solution for this equation.
  - **Good news:** The function is **concave**, meaning we can find the global maximum using iterative methods.

6.1.2 Gradient Ascent for Logistic Regression

Since the function is concave, we use the gradient to iteratively adjust the value of w toward the maximum:

- **Update Rule:**

$$w^j \leftarrow w^j + \epsilon \sum_{i=1}^{N} X_i^j \{y_i - (1 - g(X_i; w))\}$$

- **Learning Rate:** ϵ is a (small) constant representing the step size.

Gradient Ascent for Logistic Regression

Algorithm

Input

- Learning rate ϵ (a small constant).
- Initial guess for the weight vector w.

Initialize

- Choose ϵ.
- Start with an initial weight vector w.

Iterate

1. For all components j, update the weights using the gradient of the log-likelihood:

$$w^j \leftarrow w^j + \epsilon \sum_{i=1}^{N} X_i^j \{y_i - (1 - g(X_i; w))\}$$

Termination

- If there is no improvement for the Log-Likelihood (LL):

$$LL(y|X; w) = \sum_{i=1}^{N} y_i \ln(1 - g(X_i; w)) + (1 - y_i) \ln g(X_i; w)$$

- Stop; otherwise, go back to the iteration step.

Comparison: Gradient Descent vs. Gradient Ascent

Fundamental Difference The choice between these algorithms depends on the optimization objective:

- **Gradient Descent:** Used to **minimize** a function, such as a loss or error function. The algorithm moves in the opposite direction of the gradient to reach a local minimum (the "valley").
- **Gradient Ascent:** Used to **maximize** a function, such as the Log-Likelihood in Logistic Regression. The algorithm moves in the same direction as the gradient to find the local maximum (the "peak").

Update Rules The mathematical implementation is distinguished by the sign of the update step :

Gradient Ascent (for Logistic Regression)

Used when the objective function is **concave**. The **plus sign** (+) indicates moving toward the slope to increase the function value:

$$w^j \leftarrow w^j + \epsilon \frac{\partial}{\partial w^j} l(w) \tag{27}$$

Gradient Descent

Generally used when the objective function is **convex**, such as minimizing Least Squares. The **minus sign** (−) indicates moving away from the slope to decrease the function value:

$$w^j \leftarrow w^j - \epsilon \frac{\partial}{\partial w^j} \text{Loss}(w) \tag{28}$$

Feature	**Gradient Descent**	**Gradient Ascent**
Primary Goal	Minimize (Loss/Error)	Maximize (Likelihood)
Update Sign	Subtraction (−)	Addition (+)
Target Function	Convex (Bowl-shaped)	Concave (Hill-shaped)

6.2 Regularization in Logistic Regression

- **Motivation:** When we lack sufficient training samples to learn robust models, we use regularization to impose additional constraints on the parameters.
- **Probabilistic Framework:** We assume a **prior** on the parameters. For example, assuming w^i follows a Gaussian distribution with mean 0 and variance σ^2: $w^i \sim N(0, \sigma^2)$.
- **Posterior Probability:** Taking the prior into account leads to the following relationship:

$$p(y = 1, \theta | X) \propto p(y = 1 | X; \theta) p(\theta) \tag{29}$$

1. Regularized Log-Likelihood

Assuming a mean of 0 and removing terms independent of w, the Log-Likelihood (LL) is modified to include a penalty term:

$$LL(y; w|X) = \sum_{i=1}^{N} y_i w^T X_i - \ln(1 + e^{w^T X_i}) - \sum_j \frac{(w^j)^2}{2\sigma^2} \tag{30}$$

2. MAP Estimation and Update Rule

Finding the derivative with respect to w^j leads to the **MAP (Maximum A Posteriori) estimate**. The updated gradient ascent rule is:

$$w^j \leftarrow w^j + \epsilon \sum_{i=1}^{N} X_i^j \{y_i - (1 - g(X_i; w))\} - \epsilon \frac{w^j}{\sigma^2} \tag{31}$$

Where σ^2 is the variance of our prior model.

3. Types of Regularization

- **L2 Regularization:** Derived from the Gaussian prior; it minimizes the square value of w.
- **L1 Regularization:** A popular alternative that minimizes the absolute value $|w|$, often promoting sparsity.

Logistic Regression: The Logit Transformation

The name "Logistic Regression" originates from the **logit** transformation, which relates class probabilities to a linear combination of inputs:

$$\log \frac{p(y=i|X;\theta)}{p(y=k|X;\theta)} = \log \frac{g(z_i)}{g(z_k)} = w_i^0 + w_i^1 x^1 + \cdots + w_i^d x^d \tag{32}$$

6.3 Classifier Calibration

Calibration ensures that predicted class probabilities closely align with the true underlying probabilities of the sample.

- **Well-Calibrated Definition:** A model is well-calibrated if its predicted probabilities reflect the true likelihood of the event of interest.
- **Mathematical Intuition:** For a predictor h_θ on a training set of size m, a calibrated model satisfies:
$$\sum_{i=1}^{m} h_\theta(x^{(i)}) = \sum_{i=1}^{m} y^{(i)}$$
.
- **Importance of Calibration:** In scenarios like airline overbooking, an accurate individual prediction is less critical than a calibrated model that correctly estimates the total expected number of "no-show" passengers.
- **Model Comparisons:**
  * **SVM:** Generally not calibrated because the max-margin separating hyperplane predicts scores in $\mathbb{R}$ rather than probabilities ($h(x) = \langle w, x\rangle + b \in \mathbb{R}$).
  * **Logistic Regression:** Well-calibrated by construction because setting the log-loss derivative to zero directly yields the calibration identity $\sum h_\theta(x^{(i)}) = \sum y^{(i)}$.

6.3.1 Calibration Metrics and Visualization

- **Expected Calibration Error (ECE):** Discretizes predicted probabilities into B bins and calculates the weighted average difference between accuracy and confidence:
$$ECE = \sum_{b=1}^{B} \frac{n_b}{N} |acc(b) - conf(b)|$$
.
- **Reliability Diagrams:** Plots actual accuracies against confidence intervals.
  * **Overconfident:** Accuracy is lower than predicted confidence (calibration gap is below the diagonal).
  * **Underconfident:** Accuracy is higher than predicted confidence (calibration gap is above the diagonal).

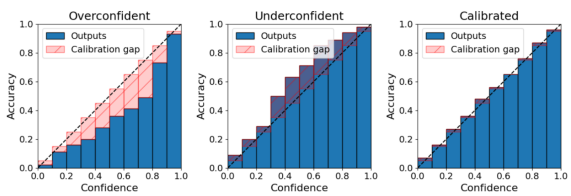


Figure 7: Reliability Diagram

Feature	Accuracy	Confidence	Calibration
Definition	The frequency with which the model predicts the correct class label.	The probability score $h_\theta(x)$ assigned by the model to a specific prediction.	The degree to which predicted class probabilities closely align with underlying probabilities of the sample.
Core Question	Is the predicted label correct?	How "sure" is the model about this specific input?	Do the probabilities reflect the true likelihood of the event of interest?
Goal	Minimize the number of individual classification errors.	Provide a score or probability for a class prediction.	Ensure the sum of predicted probabilities matches the sum of actual outcomes: $\sum h_\theta(x^{(i)}) = \sum y^{(i)}$.
Key Metric	Classification Rate / Error Rate.	Mean of the probabilities in a specific bin.	Expected Calibration Error (ECE), which measures the weighted difference between accuracy and confidence.
Visualization	Confusion Matrix or Accuracy Score.	Probability distribution / Histograms.	Reliability Diagram, which plots actual accuracies against confidence intervals.

Table 8: Comparison of Accuracy, Confidence, and Calibration based.

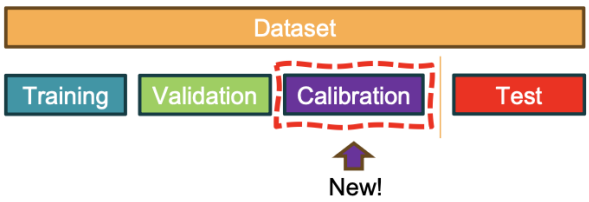
6.3.2 Calibration Methods (Post-processing)

These methods improve calibration by transforming raw model outputs $f(x)$ using a separate **calibration set** rather than the training set to avoid overfitting.

- **Platt Scaling:** Transforms model outputs through a parameterized logistic function:

$$P(y = 1|x) = \frac{1}{1 + \exp(Af(x) + B)}$$

  . Parameters A and B should be learned on a dedicated calibration set. Effective for SVM and Naïve Bayes.



- **Isotonic Regression:** A non-parametric method that transforms outputs by fitting a stepwise non-decreasing function. It minimizes the squared error $\sum_{i=1}^{n}(y_i - \hat{f}_i)^2$. While more expressive than Platt Scaling, it is more prone to overfitting and strictly requires a calibration set.